

# SAE 2.03 : Programmation d'un serveur Web configurable

## Introduction

En TP, vous avez dû programmer un serveur Web dont les fonctionnalités étaient les suivantes :

1. Reçoit des requêtes en TCP depuis un navigateur standard sur le port 80.
2. Analyse la requête pour y trouver un nom de fichier.
3. Retrouve le fichier et le renvoie par la même connexion TCP vers le navigateur, qui l'affiche.

## Évolution du serveur

Dans cette SAÉ, on va rendre ce serveur configurable, c'est à dire qu'un certain nombre de fonctionnalités devront être modifiables, sans avoir à recompiler le serveur.

Les fonctionnalités sont :

1. Le serveur pourra servir différents sites web suivant le numéro de port utilisé.
2. La spécification d'un répertoire servant de base aux sites web.
3. Retourner un fichier par défaut ou la liste des fichiers et répertoires présents (avec des liens pour naviguer).
4. Gérer des paramètres de gestion du cache.
5. Enregistrer les accès et les erreurs dans des fichiers distincts
6. La possibilité d'afficher l'état de la machine
7. Encoder les ressources en gzip.
8. Retourner les paramètres de base (Date, Server)
9. Exécuter du code « dans » une page html
10. Si certaines requêtes ne peuvent aboutir (mauvais nom de fichier, sécurité), prévoir des réponses standard (404...) pour le navigateur.

## Fichier de configuration

Le fichier de configuration doit s'appeler : serverWeb.conf

Ci-dessous un exemple de fichier de configuration :

```
<webconf>
  <site>
    port 80
    DocumentRoot /var/www
    DefaultIndex index.html
    Acceslog /var/log/myweb/access.log
    Errorlog /var/log/myweb/error.log
  </site>
  <site>
    port 8080
    DocumentRoot /var/www2
    Acceslog /var/log/myweb/access.log
```

```
Errorlog /var/log/myweb/error.log
</site>
</webconf>
```

Le serveur web doit lire ce fichier avant d'accepter des connexions. Chaque site doit avoir un numéro de port distinct. On doit pouvoir également séparer cette configuration sur plusieurs fichiers ; un pour chaque site.

## Gestion du cache

Chaque ressource doit être envoyée au client avec un champ « ETag » et un numéro unique (obtenu avec md5 ou sha). Le champ « If-Modified-since » envoyé par le client doit être pris en compte. Le serveur doit éventuellement renvoyer « Not-Modified » à la place de la ressource (cf TD).

## État du serveur

L'état de la machine sera accessible avec l'URL : `http://@serveur/status`

Il devra contenir :

- La mémoire disponible (non utilisée)
- L'espace disque disponible (idem)
- Le nombre de processus

## Encodage

Tout fichier de type image (png, jpg, pdf) sera encodé en gzip et envoyé au client. Vous devrez utiliser les champs **Content-Encoding** et **Content-Type** du protocole HTTP.

## Code dynamique

Votre serveur doit pouvoir prendre en compte la caractéristique suivante :

une balise **<code>** avec un attribut **<interpreteur>** va permettre de demander au serveur d'exécuter du code et de le remplacer dans la page html renvoyée au client.

Par exemple, la page suivante :

```
<html>
<body>
<h1> Exemple avec la date </h1>

<h2>en bash</h2>
La date est <code interpreteur=«/bin/bash»>date</code>

<h2>En python</h2>
La date est <code interpreteur=«/usr/bin/python»>
import time;
print(time.time())
</code>
```

</body>

</html>

Donne :

🔗 Most visited 📧 Getting started 📧 MAIL 📧 WEBMAIL FREE.IT

## Exemple avec la date

### en bash

La date est mardi 25 avril 2023, 11:11:05 (UTC+0200)

### En python

La date est 1682413868.99

## Programmation

Tout doit être programmé en java, avec notamment le package `java.net`. L'accès aux données du fichier de configuration doit se faire avec une API spécialisée dans la manipulation des données au format XML (`javax.xml`, `org.w3c`, ...).

## Intégration au système linux

### Fichiers

- Les classes nécessaires à votre serveur doivent être dans le répertoire :

**~/serveurWeb/bin**

- Les fichiers de configuration doivent se trouver dans le répertoire :

**~/serveurWeb/conf.d**

et leur nom doit avoir l'extension « .conf »

Quand le serveur se lance, il doit créer un fichier dans le répertoire :

**~/serveurWeb/run/**

qui doit se nommer **myweb.pid**, il contiendra le numéro du processus qui a lancé le serveur (celui de java en fait)

### Service

- Utilisez `systemD` pour créer le service « **serverWeb.service** », qui lancera ou arrêtera votre serveur web. Passez par l'intermédiaire de scripts, à créer dans **~/serveurWeb/scripts**

Pour démarrer et arrêter le service.

## **Package**

Regrouper l'ensemble de votre travail sous la forme d'un package Debian (différent d'un package java). Ce package devra contenir les classes java, le service, les scripts et fichier de configuration.

## **Travail à rendre :**

- Une description de votre programme (diagramme de classes, interactions, algo...)
- Un dossier archivé du package